

Sistema Financeiro da Casa — MVP

Documentação funcional e técnica da primeira versão

1. Visão do projeto

O Sistema Financeiro da Casa será uma aplicação web desenvolvida em Python com Flask, cujo objetivo é permitir o controle básico das finanças domésticas.

O sistema permitirá registrar receitas e despesas, organizar os lançamentos por categorias e visualizar um resumo financeiro do período.

A primeira versão será um MVP (Minimum Viable Product), portanto terá somente as funcionalidades essenciais para que o sistema seja utilizado no dia a dia.

Objetivos principais

- Registrar receitas.
- Registrar despesas.
- Organizar lançamentos por categoria.
- Consultar, editar e excluir lançamentos.
- Visualizar receitas, despesas e saldo.
- Filtrar informações por período.
- Utilizar banco de dados SQLite.
- Praticar Python, Flask e SQL durante o desenvolvimento.

2. Tecnologias

Backend — Python + Flask — lógica da aplicação, rotas, processamento das informações e comunicação com o banco.

Banco de dados — SQLite — simples, leve e sem necessidade de servidor.

Frontend — HTML + CSS + Jinja2, com JavaScript apenas quando necessário.

Controle de versão — Git + GitHub para acompanhar a evolução do projeto.

3. Escopo do MVP

```
Sistema Financeiro
├── Dashboard
├── Lançamentos
│   ├── Novo lançamento
│   ├── Editar lançamento
│   └── Excluir lançamento
├── Categorias
│   ├── Criar categoria
│   └── Listar categorias
└── Configurações básicas
```

4. Dashboard

A tela inicial deverá apresentar uma visão rápida da situação financeira: total de receitas, total de despesas, saldo, total de lançamentos, despesas por categoria e últimos lançamentos.

FINANÇAS DA CASA		
RECEITAS	DESPESAS	SALDO
R\$ 5.000,00	R\$ 3.200,00	R\$ 1.800,00

5. Lançamentos

Cada lançamento deverá possuir: ID, descrição, valor, tipo (receita ou despesa), data, categoria, observação e data de criação.

Exemplo:

Descrição: Supermercado
 Valor: 250,00
 Tipo: Despesa
 Data: 05/10/2026
 Categoria: Alimentação
 Observação: Compra da semana

6. Categorias

Categorias iniciais sugeridas:

Receitas: Salário, Freelance, Outros.

Despesas: Alimentação, Moradia, Energia, Água, Internet, Transporte, Saúde, Educação, Lazer, Compras, Outros.

O usuário poderá posteriormente criar novas categorias.

7. Banco de dados

Para o MVP, o banco terá inicialmente duas tabelas principais: categorias e lançamentos.

Tabela categorias

```
CREATE TABLE categorias (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nome TEXT NOT NULL,
  tipo TEXT NOT NULL
);
```

Tabela lançamentos

```
CREATE TABLE lancamentos (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  descricao TEXT NOT NULL,
  valor DECIMAL(10,2) NOT NULL,
  tipo TEXT NOT NULL,
  data DATE NOT NULL,
  categoria_id INTEGER,
  observacao TEXT,
  criado_em DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (categoria_id) REFERENCES categorias(id)
);
```

Relacionamento: uma categoria pode possuir vários lançamentos.

8. Regras de negócio

- O valor deve ser maior que zero.
- O tipo do lançamento determina se ele é receita ou despesa.
- Recomenda-se armazenar o valor sempre positivo e usar o campo tipo para determinar sua natureza.

- Todo lançamento deverá possuir uma categoria.
- Categorias de receita devem ser utilizadas em receitas e categorias de despesa em despesas.

Cálculo do saldo:

$SALDO = TOTAL\ DE\ RECEITAS - TOTAL\ DE\ DESPESAS$

9. Operações CRUD

Create: cadastrar lançamento. Read: consultar. Update: editar. Delete: excluir.

Novo registro → CREATE → Banco → READ → Usuário → UPDATE / DELETE

10. Páginas da aplicação

/	Dashboard
/lancamentos	Lista de lançamentos
/lancamentos/novo	Novo lançamento
/lancamentos/editar/<id>	Editar lançamento
/lancamentos/excluir/<id>	Excluir lançamento
/categorias	Lista de categorias
/categorias/nova	Nova categoria

11. Estrutura inicial do projeto

```

financeiro-casa/
├── app.py
├── database.db
├── requirements.txt
├── README.md
├── templates/
│   ├── base.html
│   ├── dashboard.html
│   ├── lancamentos.html
│   ├── novo_lancamento.html
│   ├── editar_lancamento.html
│   ├── categorias.html
│   └── nova_categoria.html
├── static/
│   ├── css/
│   │   └── style.css
│   ├── js/
│   │   └── script.js
└── database/
    └── schema.sql
  
```

12. Rotas iniciais do Flask

```

GET /
GET /lancamentos
GET /lancamentos/novo
POST /lancamentos/novo
GET /lancamentos/editar/<id>
POST /lancamentos/editar/<id>
POST /lancamentos/excluir/<id>
GET /categorias
GET /categorias/nova
POST /categorias/nova
  
```

13. Filtros

A tela de lançamentos deverá permitir filtros por período inicial/final, tipo e categoria.

14. Relatórios iniciais

O MVP terá relatórios simples: receitas x despesas e despesas agrupadas por categoria. Gráficos poderão ser adicionados depois.

15. O que NÃO fará parte do MVP

- Login e múltiplos usuários.
- Cartão de crédito e parcelamento.
- Contas bancárias e investimentos.
- Transferências entre contas.
- Importação de extrato e integração com bancos.
- Notificações e aplicativo mobile.
- Inteligência artificial.
- PostgreSQL e Docker.
- Deploy em produção.

16. Evolução futura

Versão 2: contas bancárias, cartões, parcelas, despesas recorrentes, orçamento mensal e metas.

Versão 3: login, usuários, permissões, PostgreSQL, deploy e backup.

Versão 4: API, aplicativo mobile, integração bancária, importação automática e notificações.

17. Ordem de desenvolvimento

Etapa 1 — Preparação: ambiente virtual, Flask, requirements.txt, pastas e Git.

Etapa 2 — Banco: SQLite, tabelas e categorias iniciais.

Etapa 3 — Primeiro CRUD: criar e listar lançamento.

Etapa 4 — CRUD completo: editar e excluir.

Etapa 5 — Categorias: cadastro e gerenciamento.

Etapa 6 — Dashboard: receitas, despesas, saldo e categorias.

Etapa 7 — Filtros: data, tipo e categoria.

Etapa 8 — Melhorias visuais: layout, menu, cards, tabelas, responsividade e gráficos.

18. Critério para considerar o MVP concluído

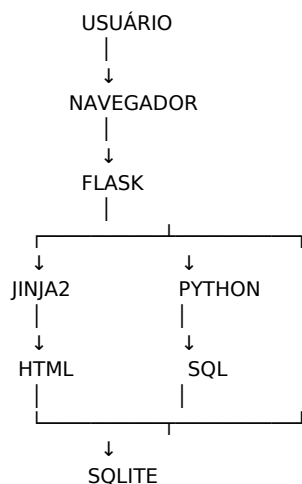
- Abrir o sistema.
- Cadastrar categoria.
- Cadastrar receita.
- Cadastrar despesa.
- Visualizar lançamentos.
- Editar lançamento.
- Excluir lançamento.
- Filtrar lançamentos.

- Visualizar total de receitas.
- Visualizar total de despesas.
- Visualizar saldo.
- Visualizar despesas agrupadas por categoria.

19. Objetivo de aprendizado

O projeto servirá para consolidar conhecimentos de Python (funções, condições, módulos e tratamento de erros), Flask (rotas, views, templates, Jinja2, formulários, GET/POST), SQL (SELECT, INSERT, UPDATE, DELETE, WHERE, ORDER BY, GROUP BY, SUM, COUNT e JOIN) e banco de dados (modelagem, relacionamentos e CRUD).

20. Visão geral da arquitetura



A aplicação será executada inicialmente de forma local, por exemplo em <http://127.0.0.1:5000>.

21. Princípio principal do projeto

Criar uma aplicação pequena, funcional e organizada, utilizando Flask, Python e SQL, e evoluí-la gradualmente conforme novos conhecimentos forem adquiridos.

Prioridade: primeiro fazer funcionar; depois organizar; depois melhorar; depois expandir.